Week 4, Day 2: Kafka Consumers in Depth, Failure Handling and Two-Heap Problems

Prepared for Upendra Kumar Sharma | 3.5 to 4 hours | Main topic: consumer internals, offsets, rebalancing, retry topics, dead letter topics, exactly-once consumption, lag | DSA: Find Median from Data Stream, Task Scheduler

Today's Schedule

- **0:00 to 0:40** Study how consumers work and how offsets and commits behave (sections 1 and 2).
- **0:40 to 1:20** Study rebalancing and parallelism (sections 3 and 4).
- **1:20 to 2:10** Study error handling: retries, retry topics, dead letter topics and idempotency (sections 5 and 6).
- **2:10 to 2:30** Study exactly-once consumption and lag (sections 7 and 8).
- **2:30 to 3:20** DSA: solve the 2 problems in section 10. Allow 25 minutes each.
- **3:20 to 3:50** Answer the interview questions aloud (section 11) and add 10 flashcards.
- **Optional (90 min):** do the hands-on experiments in section 9 and add Kafka consumers to your project.

This guide builds on Week 3, Day 5 and Week 4, Day 1. Review the consumer group and offset basics there first if needed.

1. How a Consumer Works

- A consumer **pulls** records with poll(). Each call returns a batch from the partitions assigned to it. Your code processes the batch, then calls poll() again. This loop is the heart of every consumer.
- **KafkaConsumer is not thread-safe.** Use one consumer per thread. Spring Kafka's listener containers handle this for you.
- **Group coordination:** one broker acts as the **group coordinator**. Consumers send **heartbeats** from a background thread, and the group is managed through the coordinator.
- **Key settings**
  - **max.poll.records** (500 by default): the maximum records returned per poll.
  - **max.poll.interval.ms** (5 minutes by default): the maximum time between two polls. If your processing of a batch takes longer, the consumer is **considered failed and removed from the group**, which triggers a rebalance and duplicate processing.

    - **session.timeout.ms** and **heartbeat.interval.ms:** if the coordinator receives no heartbeat within the session timeout, it removes the consumer. The heartbeat interval is usually a third of it. Defaults depend on the Kafka version, so check yours.
    - **fetch.min.bytes**, **fetch.max.wait.ms** and **max.partition.fetch.bytes:** how much data the broker returns per fetch, a throughput and latency trade-off similar to the producer's linger and batch settings.
- **Two clocks to remember:** the heartbeat thread proves the process is alive, and max.poll.interval.ms proves the **processing loop** is making progress. A consumer can be alive and still be removed because it is stuck in slow work.

2. Offsets and Commits

- The **committed offset** is the position of the **next record to read** for a partition. If you have processed offsets 0 to 41, you commit 42.
- Offsets are stored in the internal topic **__consumer_offsets**, per group, topic and partition.
- **auto.offset.reset** (earliest, latest, none) applies **only when the group has no committed offset**, or the committed offset no longer exists (for example, it was deleted by retention).

Auto commit

- enable.auto.commit=true commits the offsets from the last poll at a regular interval, in the background.
- It is easy, but it can commit records you have **not finished processing**, and a crash then **loses** them. It can also commit before work completes in a slow handler. Avoid it for anything important.

Manual commit

- **commitSync()** blocks until the broker confirms. It is safe and retries on recoverable errors.
- **commitAsync()** does not block, but it does not retry automatically, because a retry could overwrite a newer commit. A common pattern is commitAsync during normal running and **one final commitSync on shutdown or rebalance**.
- **Commit after processing** gives **at-least-once** behavior. A crash between processing and commit means the batch is processed again, so **handlers must be idempotent**.
- **Commit before processing** gives at-most-once, with possible loss.
- **Commit on revoke:** when partitions are taken away during a rebalance, commit what you have finished in the onPartitionsRevoked callback.

Replaying and repairing

- **seek()** moves a consumer to any offset, in code.
- **From the command line:** kafka-consumer-groups.sh with --reset-offsets and options such as --to-earliest, --to-datetime or --shift-by, then --execute. The group must be **inactive** while you do this.
- Replaying is a major reason to keep consumers idempotent.

3. Rebalancing

What triggers it

- A consumer joins, leaves or crashes.
- A consumer misses its session timeout, or exceeds max.poll.interval.ms.
- The partition count of a subscribed topic changes, or the subscription changes (for example, a regular-expression subscription finds a new topic).

What it costs: partitions are reassigned, and processing for the affected partitions **pauses**. Un-committed work is repeated by the new owner. Frequent rebalances (a **rebalance storm**) produce lag spikes and duplicates.

Eager vs cooperative

- **Eager (the older protocol):** every consumer gives up **all** its partitions, then the group is reassigned from scratch. The whole group pauses.
- **Cooperative (incremental):** only the partitions that must move are revoked. Others keep running. Enable it with the **CooperativeStickyAssignor**.
- **Assignors:** Range, RoundRobin, Sticky and CooperativeSticky. Sticky variants try to keep existing assignments, which reduces movement.
- Recent Kafka versions also offer a **new server-side group protocol** (KIP-848) that moves assignment work to the broker and makes rebalances less disruptive. It becomes the default direction in Kafka 4.x, so mention it as a recent improvement and check your version.

Reducing rebalances

- **Static membership:** give each instance a stable **group.instance.id**. A restart within the session timeout rejoins without triggering a rebalance. Very useful for rolling deployments and Kubernetes restarts.
- **Keep processing short** and the poll loop healthy: lower max.poll.records, speed up the handler, or raise max.poll.interval.ms when long work is unavoidable.
- **Avoid long garbage collection pauses** and CPU starvation (set container CPU limits sensibly).

- **Do not let autoscaling flap.** Scale on stable signals such as lag, with a cool-down period.
- **Roll deployments slowly** so the group is not repeatedly reshuffled.

4. Parallelism and Back-Pressure

- **The unit of parallelism is the partition.** A group can use at most as many active consumers as partitions. Plan partition counts for your peak need.
- **Spring concurrency:** setting concurrency to 3 starts 3 consumer threads in one instance, each with its own consumer. Keep the total across all instances at or below the partition count.
- **Handing records to a thread pool breaks ordering and makes offset management tricky** (you must not commit an offset until every earlier record is done). If you need more parallelism than partitions allow, increase the partitions, or process in parallel by **key** with one ordered queue per key group, and commit only up to the lowest completed offset.
- **Batch listeners:** process a list of records at once to make database writes more efficient (for example, one multi-row insert). Handle partial failure carefully.
- **pause() and resume()** stop and restart fetching for some partitions without leaving the group. Use them for **back-pressure** when a downstream dependency is slow or a circuit breaker is open, instead of letting records pile up in memory.

5. Error Handling Strategy

First, classify the failure

- **Transient** (database timeout, downstream 503, lock timeout): retrying may succeed.
- **Permanent** (malformed payload, validation failure, unknown event version, a business rule that can never pass): retrying is pointless and only delays everything behind it.
- **Poison pill:** a record that fails every time, often on **deserialization**. It can block a partition forever if you handle it badly.

Option 1: blocking retry in place (preserves order)

- Spring's **DefaultErrorHandler** retries the failed record with a back-off, **blocking that partition**, and when retries are exhausted it passes the record to a **recoverer**, usually a dead letter publisher.
- Good when **order matters** and failures are short-lived. Bad for long outages, because the partition stalls and lag grows.

Option 2: non-blocking retry topics (keeps the partition moving)

- The failed record is published to a **retry topic** with a delay, and the main consumer continues. After the maximum attempts it goes to the **dead letter topic**.
- In Spring Kafka this is **@RetryableTopic**, which creates topics such as orders.created-retry-0, orders.created-retry-1 and orders.created-dlt.
- The trade-off: the record is processed **later and out of order** relative to other records for the same key. Use it only when events are independent or the handlers tolerate reordering.

```
@Component
public class OrderCreatedListener {

    @RetryableTopic(
        attempts = "4",
        backoff = @Backoff(delay = 1000, multiplier = 2.0),
        include = TransientProcessingException.class)
    @KafkaListener(topics = "orders.created", groupId = "inventory-service")
    public void handle(OrderCreated event) {
        inventoryService.reserve(event);       // idempotent; throws
TransientProcessingException on timeouts
    }

    @DltHandler
    public void dead(OrderCreated event, @Header(KafkaHeaders.RECEIVED_TOPIC)
String topic) {
        // record, alert, and keep the payload for a later redrive
    }
}
```

Blocking handler for ordered flows (from Week 3, Day 5)

```
@Bean
DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    var recoverer = new DeadLetterPublishingRecoverer(template);
    var handler = new DefaultErrorHandler(recoverer, new FixedBackOff(1000L,
3L));
    handler.addNotRetryableExceptions(IllegalArgumentException.class,
ValidationException.class);
    return handler;
}
```

Deserialization errors: wrap your deserializer with **ErrorHandlingDeserializer**, so a bad record becomes a handled error and goes to the DLT instead of crashing the container in an endless loop.

6. Dead Letter Topics and Idempotent Consumers

Dead letter topic (DLT) practice

- A DLT is **not a trash can.** Each message in it is a business event that did not happen. Treat it as an operational queue.
- **Keep context:** the original topic, partition, offset, and the exception message and class (Spring's recoverer adds these as headers).
- **Alert** on any new DLT message, or on a rising count.
- **Redrive:** build a safe way to replay DLT messages to the main topic after the bug is fixed. Because redrive repeats events, **handlers must be idempotent**.
- **Retention:** keep DLT data longer than the main topic, so there is time to investigate.
- **Do not retry endlessly.** After a limited number of attempts, move on and let humans decide.

Idempotent consumer (the inbox pattern from Week 3, Day 4)

```
@Transactional
public void reserve(OrderCreated event) {
    int inserted = processed.insertIfNew(event.eventId());   // INSERT ... ON
CONFLICT DO NOTHING
    if (inserted == 0) return;                                // duplicate
delivery: ignore it

    for (Item item : event.items()) {
        int updated = stock.decrementIfAvailable(item.sku(),
item.quantity());
        if (updated == 0) {
            outbox.save(OutboxEvent.of("Order", event.orderId(),
"InventoryRejected", reasonJson(item)));
            return;                                           // roll back
partial reservations as needed
        }
    }
    outbox.save(OutboxEvent.of("Order", event.orderId(), "InventoryReserved",
payloadJson(event)));
}
```

- The dedupe record, the stock changes and the outbound event all commit **in one database transaction**, so a crash leaves nothing half done. In a real implementation, make sure a partial reservation is undone when one item fails (for example, by throwing and rolling back the transaction, then recording the rejection in a separate step).

- A conditional update (UPDATE ... WHERE available >= :qty) is the safe way to avoid overselling under concurrency.

7. Exactly-Once Consumption

Inside Kafka (consume, process, produce)

- The consumer reads with **isolation.level=read_committed**, and the producer writes the results and the **input offsets** in one **transaction** (sendOffsetsToTransaction). Either the outputs and the offset commit all happen, or none do.
- In Spring Kafka, set a **transaction-id-prefix** on the producer, and the listener container runs each record or batch inside a transaction.
- This is how stream processing pipelines get exactly-once results **between topics**.

When a database is involved

- The Kafka transaction does **not** include your database. Choose one of:
  - **Idempotent consumer** (the inbox table above): at-least-once delivery plus deduplication gives effectively-once results. The most common choice.
  - **Store offsets in the database** together with the results, in one database transaction, and on startup **seek** to the stored offsets. This avoids duplicates entirely but needs more code.
- State clearly in interviews: **Kafka provides exactly-once within Kafka. For external side effects, you provide idempotency.**

8. Monitoring Consumer Lag

- **Lag** is the difference between the newest offset in a partition and the group's committed offset. It tells you how far behind the group is.
- **Check it:** kafka-consumer-groups.sh --describe --group inventory-service, or Micrometer metrics from the client (for example records-lag-max), or a tool such as Burrow or a Kafka exporter in Prometheus.
- **Alert on trends, not just thresholds.** Lag that grows steadily means consumers are slower than producers. Also watch **time lag** (how old the oldest unprocessed record is).
- **If lag keeps growing**
  1. Look for rebalance loops, errors and retries first. They are common hidden causes.
  2. Profile the handler. Slow database queries and downstream calls are usual culprits.
  3. Use batch processing and bulk database operations.
  4. Add consumers, up to the partition count.

5. Add partitions (aware of the key ordering caveat), and consider pausing low-priority work.
6. Protect the downstream dependency with a bulkhead or circuit breaker (Week 3, Day 3), and use pause() as back-pressure.

9. Optional Hands-On (90 Minutes)

1. **Manual acknowledgement.** Set ack-mode to manual. Process a record, then kill the application **before** acknowledging. Restart and see the same record again. This is at-least-once in action.
2. **Rebalance observation.** Run 2 instances, then add a third, then kill one. Note which partitions move and how long processing pauses. Repeat with a **group.instance.id** set and a quick restart, and compare.
3. **max.poll.interval.ms.** Set it to 10 seconds and add a 15-second sleep to the handler. Watch the consumer get kicked out and the batch repeated. Then lower max.poll.records and see the effect.
4. **Poison message.** Produce an invalid JSON string. With and without ErrorHandlingDeserializer, observe the difference (an endless error loop against a DLT entry).
5. **Retry topics.** Make the handler throw a transient exception for the first two attempts. Use @RetryableTopic and watch the retry topics fill and the message succeed on the third try. Then make it always fail and find the message in the DLT.
6. **Lag.** Stop the consumers, produce 10,000 messages, and run the group describe command to see the lag. Start a consumer and watch it fall.
7. **Offset reset.** With the consumers stopped, reset the group to earliest and watch all messages replay. Notice that the idempotent handler ignores them.
8. **Project step.** In inventory-service, add the consumer for orders.created with the inbox table and the atomic stock update, publishing InventoryReserved or InventoryRejected through its own **outbox**. In order-service, consume those events and set the order to CONFIRMED or CANCELLED. Add notification-service as a consumer of the order outcome events. Deliver one event twice in a test and confirm the effect happens once.

Keep notes of what you observed. They turn into precise, confident answers.

10. DSA: Two-Heap and Greedy Problems (2 Problems)

Problem 1: Find Median from Data Stream

- **Idea:** keep the numbers split into two halves with **two heaps**: a **max-heap** holding the smaller half and a **min-heap** holding the larger half. Every number in the lower half is less than or equal to every number in the upper half, and the sizes differ by at most 1. The median is the top of the larger heap, or the average of both tops when the sizes are equal.
- **Adding a number:** push it into the max-heap, move the max-heap's top into the min-heap (this keeps the ordering rule), and if the min-heap is now larger, move its top back to the max-heap.
- **Complexity:** addNum is O(log n), findMedian is O(1), and space is O(n).
- **Edge cases:** a single number, duplicates, negative numbers, and large values (add them as long before halving, to avoid overflow).
- **Follow-ups to prepare:** if all numbers are in the range 0 to 100, use a counting array and find the median by cumulative counts in O(1) space per query. If 99 percent are in that range, keep counts for the range and a small structure for outliers.

```
class MedianFinder {
    private final PriorityQueue<Integer> lower = new PriorityQueue<>
(Comparator.reverseOrder());
    private final PriorityQueue<Integer> upper = new PriorityQueue<>();

    public void addNum(int num) {
        lower.offer(num);
        upper.offer(lower.poll());
        if (upper.size() > lower.size()) lower.offer(upper.poll());
    }

    public double findMedian() {
        if (lower.size() > upper.size()) return lower.peek();
        return ((long) lower.peek() + upper.peek()) / 2.0;
    }
}
```

Problem 2: Task Scheduler

- **The problem:** tasks are letters. The same task needs at least n intervals of cooldown between two runs. Return the minimum number of intervals (idle slots included) to finish all tasks.

- **Idea:** the task with the highest frequency decides the layout. Arrange it into (maxFreq minus 1) **frames** of size (n + 1), then a final partial frame that holds every task tied at that maximum frequency. The length is (maxFreq - 1) x (n + 1) + countOfMaxFreq. If other tasks fill the idle gaps, the answer is simply the **number of tasks**, so take the larger of the two.
- **Complexity:** O(T) time for T tasks, O(1) space (26 letters).
- **Edge cases:** n equal to 0 (the answer is the task count), only one kind of task, many tasks tied at the maximum.
- **Alternative:** a **max-heap** of remaining counts plus a cooldown queue simulates the schedule step by step in O(T log 26). Be ready to describe it.
- **Connection to systems:** the same cooldown idea appears in retry back-off and rate limiting.

```
public int leastInterval(char[] tasks, int n) {
    int[] freq = new int[26];
    for (char c : tasks) freq[c - 'A']++;

    int maxFreq = 0;
    for (int f : freq) maxFreq = Math.max(maxFreq, f);

    int countMax = 0;
    for (int f : freq) if (f == maxFreq) countMax++;

    return Math.max(tasks.length, (maxFreq - 1) * (n + 1) + countMax);
}
```

11. Interview Questions to Answer Aloud

1. Describe the consumer poll loop. Why is KafkaConsumer not thread-safe?
2. What does a committed offset mean? What is the difference between committing before and after processing?
3. Why is auto commit risky? When would you use commitSync and commitAsync?
4. What triggers a rebalance, and what does it cost? How do you reduce rebalances (static membership, cooperative assignor)?
5. What happens when processing exceeds max.poll.interval.ms?
6. How do you scale a consumer group, and what is the ceiling?
7. Why does handing records to a thread pool cause problems for ordering and offsets? How would you get more parallelism safely?
8. How do you classify failures, and what do you do with transient vs permanent errors?
9. Blocking retry vs retry topics: what are the trade-offs, especially for ordering?

10. What is a dead letter topic? What should be in it, and how do you redrive messages?
11. How do you handle a poison pill message that fails to deserialize?
12. How do you achieve effectively-once processing when your handler writes to a database?
13. What does exactly-once mean in Kafka, and what does it not cover?
14. Consumer lag keeps growing. Walk me through how you would find and fix the cause.

12. End-of-Day Checklist

- [ ] I can explain the poll loop, the two timeouts and what each protects.
- [ ] I can explain committed offsets, auto vs manual commit, and why handlers must be idempotent.
- [ ] I can explain the causes and costs of rebalances and at least four ways to reduce them.
- [ ] I can design error handling: classification, blocking retries, retry topics and a DLT with a redrive plan.
- [ ] I can implement an idempotent consumer with an inbox table and an outbox event in one transaction.
- [ ] I can explain exactly-once inside Kafka and what is left to me for databases.
- [ ] I can diagnose growing lag step by step.
- [ ] I solved both DSA problems and can explain the two-heap invariant and the frame formula.
- [ ] (Optional) My project now has consumers, the inbox and the outbox working end to end, and I tested a duplicate delivery.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 4, Day 3): Redis data structures and caching patterns (cache-aside, write-through, TTL, eviction, stampede), plus binary search on the answer problems.